MegaSplonk

Modular Split Keyboard

Monday 01 June 2026

Table of Contents

Features 3
Splonk 3
Splonkule 3
Overall 3
Hight level explanation 4
Base concept 4
Modularity 4
Topology 4
Input device specifics 4
Functional Schemas 5
User Guide 7
Bill of Materials 8
Power Draw 10
Splonk 11
Splonkule 13
Overall 15
Pinout 16
Protocols 17
I²C 17
SPI 19
USB 21
Final Words 24
Erratum 24
Splonkpad 24

Features

Splonk

A *Splonk* is a keyboard half. A full keyboard is thus called a *MegaSplonk*.

Splonker

The *Splonker* is the *Splonk* that handles communication with the computer, i.e. the one with an active USB connection. It can accommodate *Splonkules*.

Splonkee

The *Splonkee* is the other *Splonk*, connected to the *Splonker* (and possibly some *Splonkules*).

Splonkule

You can attach modules that add functionality to a *Splonk*, *Splonkules*, if you will.

Splonkus

The *Splonkus* is a *Splonk* that behaves like a *Splonkule*, meaning it *cannot* accommodate its own *Splonkules*. You can press it.

Splonknob

The *Splonknob* is a rotary encoder input device. You can spin it.

***Splonkbar* (optional)**

The *Splonkbar* is a sliding encoder input device. You can slide it.

Splonkpad

The *Splonkpad* is a capacitive input device. You can touch it.

Splonkreen

The *Splonkreen* is a programmable mini screen. You can see it.

Splonko

The *Splonko* is a small keyboard addon. You can press it.

Overall

To use a *MegaSplonk* you only need a *Splonker*. The *Splonker* can accommodate up to 4 modules.
You can also connect another *Splonker* to the first one, making it a *Splonkee*, as it won't handle any USB or Module connection.
This makes it possible to have up to 6 modules, since you need to use a module port on each *Splonk* to connect them.
Any module can be connected to any port, and they will be recognised.
Beware that some module will draw more current than other, and you can draw a fixed amount of current from your computer. This means that for example if you try to use all six module slots with a high power module, it wont work. This may render your *Splonker* unusable until you change the fuse
Some modules might have limited functionality if the power limit is reached (for example LEDS might not work). This is expected and wanted to avoid blowing fuses too easily. Treat it as a warning !

Hight level explanation

Base concept

The *MegaSplonk* keyboard is a split keyboard with a module system. Being a split keyboard, it is intended to have two halves that are connected by a cable.
You only need to connect a single half to the computer using USB for it to work.

The main appeal of this keyboard over others is the module system. This allows the user to customize the product without having to touch the mainboards.

Modularity

The modules are either connected to one half using magnets next to the connector, meaning they are right next to it, or they connect to the half using a special made cable. Either way they all communicate using the same protocol (I^2C) to the *Main* board (The one connected to the computer via USB).

The mainboard then processes everything and sends the data to the computer using the standard USB HID (*Human Interface Device*). USB HID is a standard that doesn't require any user to install a driver on their computer.

Topology

The modules use a *tiny* micro controller that enables them to speak the same *language* (Over I^2C) thus permitting the addition of modules easily.
The keyboard half can have two roles: the *master* and the *slave*
The *slave* is used as another module, albeit with a more powerful micro controller since it *could* be a *master* at some point.
They still retain the role of managing the keyboard presses and LEDs on their side.
The *master* has far more responsibility:

- It must talk to every module to gather the inputs
- It must handle USB
- It also shares the same responsibility as the *slave*, meaning it manages its own key presses and LEDs

The Atmega32U4 has dedicated USB protocol support, making its use somewhat mandatory with our needs.

Every modules that uses LEDs will use the same LEDs chip, and each LED "zone" is managed using SPI.

Input device specifics

Any board that needs to register keypresses from the user will be done in a way that wont skip over keypresses, or create weird inputs when multiple are pressed at once.
This is called **N-KRO** for "N Key roll over". This is handled on the device itself and not by the computer. Some very cheap keyboards do not have this basic feature.

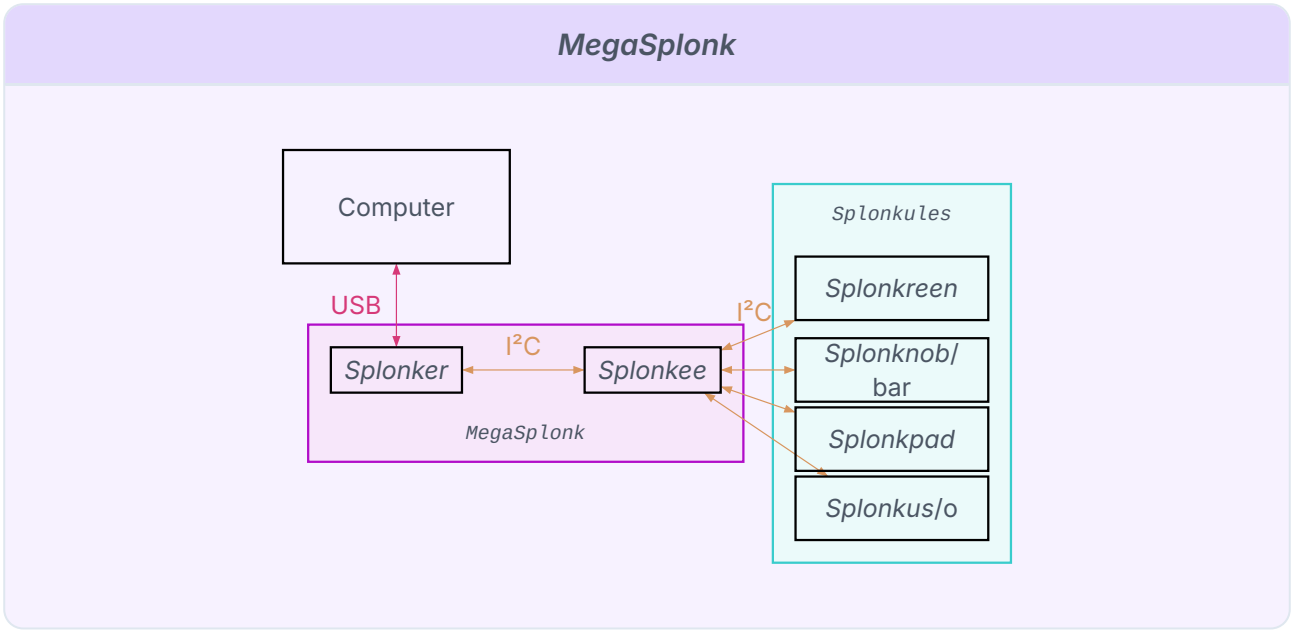
For some modules, we will have to implement other "virtual devices" (not keyboards). For example the touchpad module will emulate a mouse.
This must be done at the USB level, by having multiple Endpoints (think of them as another pipe in a larger USB connection pipe).

These endpoints can be a multitude of things, but having a “mouse” endpoint would allow us to send commands to the computer that moves the cursor depending on the touchpad’s capacitive inputs.

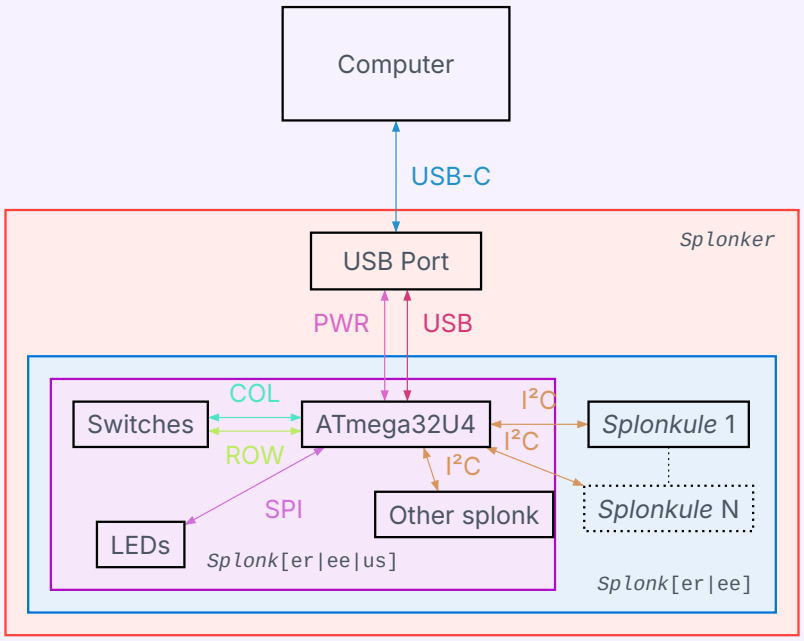
The keyboard layout and configuration will be done by having the user upload a new program to the *master* microcontroller, thus reducing the latency by not having to interpret any configuration data.
This will also allow greater customization.

It may be possible to upload such program directly through the USB connection.

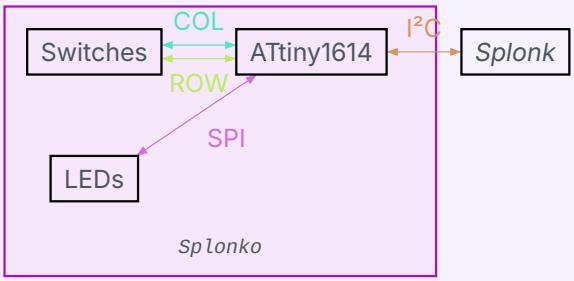
Functional Schemas



Splonker / *Splonkee* / *Splonkus*



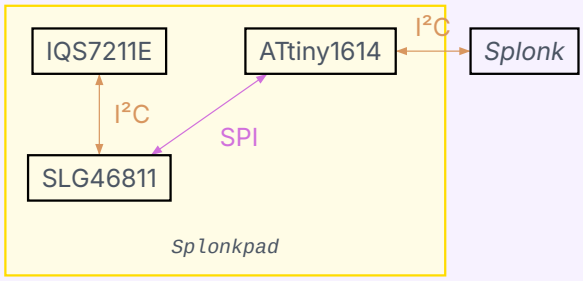
Splonko



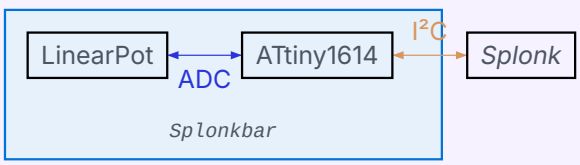
Splonkreen

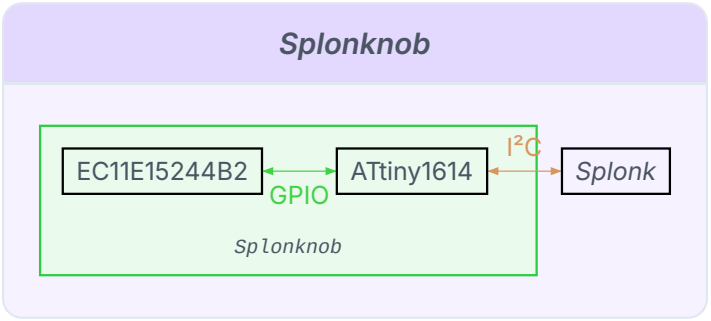


Splonkpad



Splonkbar (optional)





User Guide

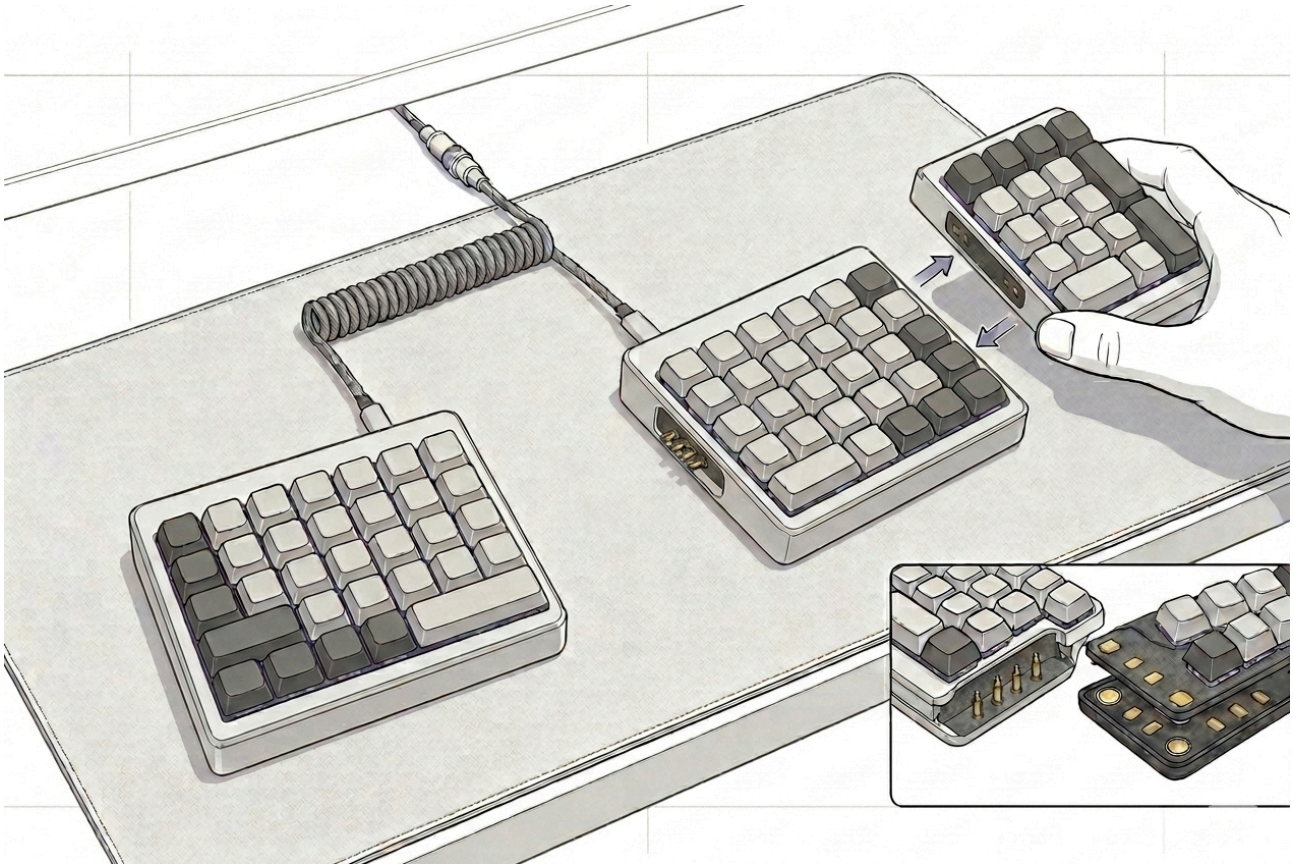


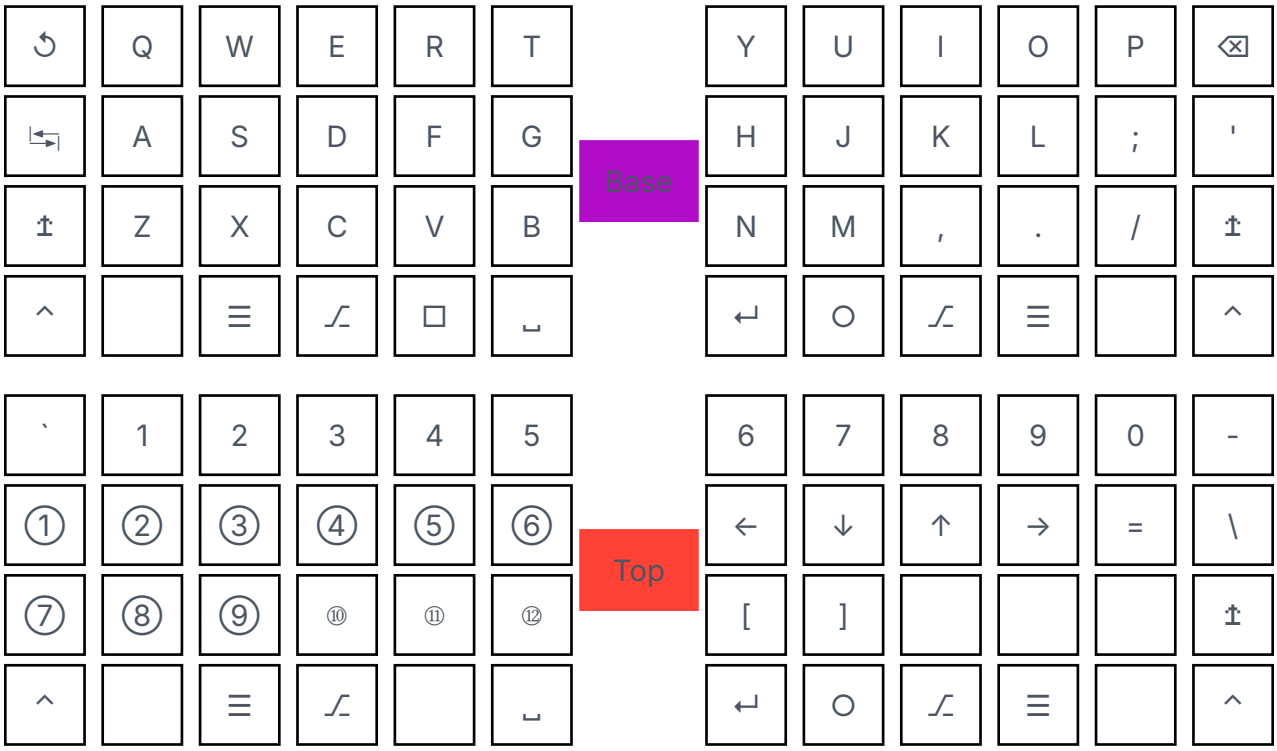
Figure 1: Non contractual representation of a *MegaSplonk* in use

As a *MegaSplonk* user, you have total freedom in both physical and firmware configuration. You can connect any *Splonkule* of your choosing to any *Splonk*, and make any key press output any desired character on your computer.

The former is achieved by simply *Splonking* the components together, as they will hold strong through magnetic bonds.

The latter can be done with firmware configuration, a common way of using smaller/split keyboards while having as many or more keys as a traditional keyboard is the use of **layers**, accessed with a modifier key. Think of it like the shift key allowing you to access special characters on the numbers row, or the fn key for the top row shortcuts, but for the whole keyboard!

Here's an example of layout and layers one could use on their *MegaSplonk* keyboard:



In this example the user can access the Top layer by pressing the ☐ (top) key.

A typical split keyboard user defines 2 to 4 layers, but many firmwares allow for many more, the only limit is your imagination!

Bill of Materials

Name	Count	Unit	Total	Datasheet	Link	Desc
ATmega32U4	3	5.58€	16.74€			USB capable AVR MCU
Apa102	78	0.06€	4.55€			LEDs over SPI
Mx Socket	78	0.13€	10.34€	*N/A*		Cherry MX compatible hotswappable key socket (150piece)
Diode	78	*N/A*	*N/A*	*N/A*	*N/A*	
PogoFemale	9	0.75€	6.75€	*N/A*		5 Pin Pogo Connector Female
USB C	2	0.13€	0.26€	*N/A*		USB C female connector
Cap 1uF	10	0€	0€	*N/A*	*N/A*	
Cap 100nF	26	0€	0€	*N/A*	*N/A*	
Cap 10uF	4	0€	0€	*N/A*	*N/A*	
Total			**68.41€**	*per unit*		

Name	Count	Unit	Total	Datasheet	Link	Desc
Cap 27pF	6	0.45€	2.67€	*N/A*		
Cap elec 100uF	7	0€	0€	*N/A*	*N/A*	
Diode Schottky	2	0€	0€	*N/A*	*N/A*	
Polyfuse	2	0€	0€	*N/A*	*N/A*	
LED	6	0.09€	0.52€	*N/A*		SK9822
Res 5.1k Ohm	4	0€	0€	*N/A*	*N/A*	
Res 22 Ohm	4	0€	0€	*N/A*	*N/A*	
Res 10k Ohm	20	0€	0€	*N/A*	*N/A*	
Res 4.7k Ohm	4	0€	0€	*N/A*	*N/A*	
Reset Switch	3	0€	0€	*N/A*	*N/A*	
TS5A23157DGSR	8	*N/A*	*N/A*	*N/A*	*N/A*	Analogue switch
ATtiny1614	6	1.89€	11.33€			Small microcontroller
PogoMale	5	0.96€	4.8€	*N/A*		5 Pin Pogo Connector Male
EC11E15244B2	2	2€	4€			Rotary encoder
Cap 10nF	11	*N/A*	*N/A*	*N/A*	*N/A*	
Res 10K	12	*N/A*	*N/A*	*N/A*	*N/A*	
Pot	1	*N/A*	*N/A*	*N/A*	*N/A*	potentiometre
IQS7211E	1	1.24€	1.24€			Capacitive touch controller
Cap 2.2uF	2	*N/A*	*N/A*	*N/A*	*N/A*	
Cap 4.7uF	1	*N/A*	*N/A*	*N/A*	*N/A*	
Cap 100pF	3	*N/A*	*N/A*	*N/A*	*N/A*	
Cap 100uF	1	*N/A*	*N/A*	*N/A*	*N/A*	
TXS0108EPW	1	*N/A*	*N/A*	*N/A*	*N/A*	Level Shifter
Res 10k	4	*N/A*	*N/A*	*N/A*	*N/A*	
Res 4.7k	4	*N/A*	*N/A*	*N/A*	*N/A*	
AMS1117-3.3	1	*N/A*	*N/A*	*N/A*	*N/A*	
SH1106	1	5.2€	5.2€	*N/A*		Generic screen driver
Total			**68.41€**	*per unit*		

Name	Count	Unit	Total	Datasheet	Link	Desc
						(and screen module)
Total			**68.41€**	*per unit*		

ℓ We wanted to trick **JLCPCB** by merging multiple PCBs into one (they are too smart - we gave up)

Pcb	Dimensions	Price	Comment
Splonk	*85mm x 250mm*	14€	2 PCB in one
Splonkules	*N/A*	14€	Every *Splonkule* PCB in one
Total		**28€**	

Power Draw

G This hasn't been updated to reflect the final product power draw

Splonk

Splonker

Component	Count	Avg	Max	Total Avg	Total Max
ATmega32U4	1	27mA	30mA	27mA	30mA
Apa102	24	2mA	20mA	48mA	480mA
Mx Socket	24	0mA	0mA	0mA	0mA
Diode	24	*N/A*	*N/A*	*N/A*	*N/A*
PogoFemale	4	0mA	0mA	0mA	0mA
USB C	1	0mA	0mA	0mA	0mA
Cap 1uF	3	0mA	0mA	0mA	0mA
Cap 100nF	9	0mA	0mA	0mA	0mA
Cap 10uF	1	0mA	0mA	0mA	0mA
Cap 27pF	2	0mA	0mA	0mA	0mA
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Diode Schottky	1	0mA	0mA	0mA	0mA
Polyfuse	1	0mA	0mA	0mA	0mA
LED	2	0mA	0mA	0mA	0mA
Res 5.1k Ohm	2	0mA	0mA	0mA	0mA
Res 22 Ohm	2	0mA	0mA	0mA	0mA
Res 10k Ohm	8	0mA	0mA	0mA	0mA
Res 4.7k Ohm	2	0mA	0mA	0mA	0mA
Reset Switch	1	0mA	0mA	0mA	0mA
TS5A23157DGSR	4	*N/A*	*N/A*	*N/A*	*N/A*
Total				**75mA**	**510mA**

Splonkee

Component	Count	Avg	Max	Total Avg	Total Max
ATmega32U4	1	27mA	30mA	27mA	30mA
Apa102	24	2mA	20mA	48mA	480mA
Mx Socket	24	0mA	0mA	0mA	0mA
Diode	24	*N/A*	*N/A*	*N/A*	*N/A*
PogoFemale	4	0mA	0mA	0mA	0mA
USB C	1	0mA	0mA	0mA	0mA
Cap 1uF	3	0mA	0mA	0mA	0mA
Cap 100nF	9	0mA	0mA	0mA	0mA
Cap 10uF	1	0mA	0mA	0mA	0mA
Cap 27pF	2	0mA	0mA	0mA	0mA
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Diode Schottky	1	0mA	0mA	0mA	0mA
Polyfuse	1	0mA	0mA	0mA	0mA
LED	2	0mA	0mA	0mA	0mA
Res 5.1k Ohm	2	0mA	0mA	0mA	0mA
Res 22 Ohm	2	0mA	0mA	0mA	0mA
Res 10k Ohm	8	0mA	0mA	0mA	0mA
Res 4.7k Ohm	2	0mA	0mA	0mA	0mA
Reset Switch	1	0mA	0mA	0mA	0mA
TS5A23157DGSR	4	*N/A*	*N/A*	*N/A*	*N/A*
Total				**75mA**	**510mA**

Splonkule

Splonkus

Component	Count	Avg	Max	Total Avg	Total Max
ATmega32U4	1	27mA	30mA	27mA	30mA
Apa102	24	2mA	20mA	48mA	480mA
Mx Socket	24	0mA	0mA	0mA	0mA
Diode	24	*N/A*	*N/A*	*N/A*	*N/A*
PogoFemale	1	0mA	0mA	0mA	0mA
Cap 1uF	3	0mA	0mA	0mA	0mA
Cap 100nF	5	0mA	0mA	0mA	0mA
Cap 10uF	1	0mA	0mA	0mA	0mA
Cap 27pF	2	0mA	0mA	0mA	0mA
LED	2	0mA	0mA	0mA	0mA
Res 10k Ohm	4	0mA	0mA	0mA	0mA
Reset Switch	1	0mA	0mA	0mA	0mA
Total				**75mA**	**510mA**

Splonknob

Component	Count	Avg	Max	Total Avg	Total Max
ATtiny1614	1	5mA	10mA	5mA	10mA
PogoMale	1	0mA	0mA	0mA	0mA
EC11E15244B2	2	10mA	10mA	20mA	20mA
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Cap 10nF	7	*N/A*	*N/A*	*N/A*	*N/A*
Cap 100nF	1	0mA	0mA	0mA	0mA
Cap 1uF	1	0mA	0mA	0mA	0mA
Res 10K	12	*N/A*	*N/A*	*N/A*	*N/A*
Total				**25mA**	**30mA**

Splonkbar (optional)

Component	Count	Avg	Max	Total Avg	Total Max
ATtiny1614	1	5mA	10mA	5mA	10mA
PogoMale	1	0mA	0mA	0mA	0mA
Pot	1	0mA	0mA	0mA	0mA
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Cap 10nF	1	*N/A*	*N/A*	*N/A*	*N/A*
Total				**5mA**	**10mA**

Splonkpad

Component	Count	Avg	Max	Total Avg	Total Max
ATtiny1614	2	5mA	10mA	10mA	20mA
IQS7211E	1	6mA	30mA	6mA	30mA
PogoMale	1	0mA	0mA	0mA	0mA
Cap 2.2uF	2	*N/A*	*N/A*	*N/A*	*N/A*
Cap 4.7uF	1	*N/A*	*N/A*	*N/A*	*N/A*
Cap 100pF	3	*N/A*	*N/A*	*N/A*	*N/A*
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Cap 10nF	1	*N/A*	*N/A*	*N/A*	*N/A*
Cap 100nF	2	0mA	0mA	0mA	0mA
Cap 100uF	1	*N/A*	*N/A*	*N/A*	*N/A*
TXS0108EPW	1	*N/A*	*N/A*	*N/A*	*N/A*
Res 10k	2	*N/A*	*N/A*	*N/A*	*N/A*
Res 4.7k	4	*N/A*	*N/A*	*N/A*	*N/A*
AMS1117-3.3	1	*N/A*	*N/A*	*N/A*	*N/A*
Total				**16mA**	**50mA**

Splonkreen

Component	Count	Avg	Max	Total Avg	Total Max
ATtiny1614	1	5mA	10mA	5mA	10mA
SH1106	1	20mA	50mA	20mA	50mA
PogoMale	1	0mA	0mA	0mA	0mA
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Cap 10nF	1	*N/A*	*N/A*	*N/A*	*N/A*
Total				**25mA**	**60mA**

Splonko

Component	Count	Avg	Max	Total Avg	Total Max
ATtiny1614	1	5mA	10mA	5mA	10mA
Apa102	6	2mA	20mA	12mA	120mA
Mx Socket	6	0mA	0mA	0mA	0mA
Diode	6	*N/A*	*N/A*	*N/A*	*N/A*
PogoMale	1	0mA	0mA	0mA	0mA
Cap elec 100uF	1	0mA	0mA	0mA	0mA
Cap 10nF	1	*N/A*	*N/A*	*N/A*	*N/A*
Cap 10uF	1	0mA	0mA	0mA	0mA
Res 10k	2	*N/A*	*N/A*	*N/A*	*N/A*
Total				**17mA**	**130mA**

Overall

Name	Max Power Draw	Avg Power Draw
Splonker	510mA	75mA
Splonkee	510mA	75mA
Splonkus	510mA	75mA
Splonknob	30mA	25mA
***Splonkbar* (optional)**	10mA	5mA
Splonkpad	50mA	16mA
Splonkreen	60mA	25mA
Splonko	130mA	17mA
Total	**1810mA**	**313mA**

G The LEDs can draw lots of current, way more than what we can ever get out of our simple USB connector. But they only draw a lot of current if you put them at max brightness, which we won't do as we would like to keep our eyesight.

The overall power budget is **500mA** since we source our power from USB, which provides 5V and only 0.5A. It is possible to draw more current with a USB-C connector, but it would require USB-PD which is not something a keyboard should need!

Pinout

Splonker / *Splonkee* / *Splonkus*

	#	Name	Description
USB	2	UVcc	USB UVcc
	3	D-	USB D-
	4	D+	USB D+
	5	UGnd	USB Gnd
	6	UCap	USB Cap
	7	VBus	USB VBus
SPI	9	PB1	SPI Clock
	10	PB2	SPI Data (M→S)
	11	PB3	SPI Data (S→M)
I²C	18	PD0	I²C Clock
	19	PD1	I²C Data
COL	25	PD4	Key Col 1
	26	PD6	Key Col 2
	27	PD7	Key Col 3
	28	PB4	Key Col 4
	29	PB5	Key Col 5
	30	PB6	Key Col 6
ROW	38	PF5	Key Row 1
	39	PF4	Key Row 2
	40	PF1	Key Row 3
	41	PF0	Key Row 4
UART	20	PD2	RX
	21	PD3	TX
BUS ENABLE	31	PC6	BE1
	22	PD5	BE2
	12	PB7	BE3
	1	PE6	BE4

Splonknob

	#	Name	Description
GPIO	11	PA1	B1
	12	PA2	A1
	13	PA3	S1
	2	PA4	S2
	3	PA5	A2
	4	PA6	B2
I²C	8	PB1	I²C Data
	9	PB0	I²C Clock
UART	7	PB2	RX
	6	PB3	TX

Splonkbar (optional)

	#	Name	Description
ADC	3	PA5	Pot
I²C	8	PB1	I²C Data
	9	PB0	I²C Clock
UART	7	PB2	RX
	6	PB3	TX

Splonkpad

	#	Name	Description
Soft I²C	2	PA4	SCL
	3	PA5	SDA
	4	PA6	RDY
I²C	8	PB0	I²C Clock
	9	PB1	I²C Data
UART	7	PB2	RX
	6	PB3	TX

Splonko

	#	Name	Description
ROW	12	PA2	Key Row 1
	2	PA4	Key Row 2
COL	3	PA5	Key Col 1
	4	PA6	Key Col 2
	5	PA7	Key Col 3
software SPI	11	PA1	SPI Data (M→S)
	13	PA3	SPI Clock
I²C	9	PB0	I²C Clock
	8	PB1	I²C Data
UART	7	PB2	RX
	6	PB3	TX

Splonkreen

	#	Name	Description
SPI	11	PA1	SPI Data (M→S)
	12	PA2	SPI Data (S→M)
	13	PA3	SPI Clock
	2	PA4	SPI Slave Select
I²C	8	PB0	I²C Clock
	9	PB1	I²C Data
UART	7	PB2	RX
	6	PB3	TX

Protocols

I²C

I²C is a synchronous serial communication bus used for communication between integrated circuits on the same PCB. It requires only two shared signal lines:

SCL (Serial Clock) — provides the timing reference. SDA (Serial Data) — carries address and data information.

Multiple devices can be connected to the same bus. One device acts as the controller and initiates communication, while one or more targets respond when their assigned address is selected.

Electrical Operation

Unlike many digital interfaces, I²C uses an open-drain signaling scheme. Devices connected to the bus can actively pull a signal line to a LOW state, but they do not drive it HIGH. Instead, pull-up resistors return the lines to the HIGH state when no device is pulling them down. This arrangement allows multiple devices to safely share the same wires without electrical conflicts.

When the bus is idle, both signals remain HIGH.

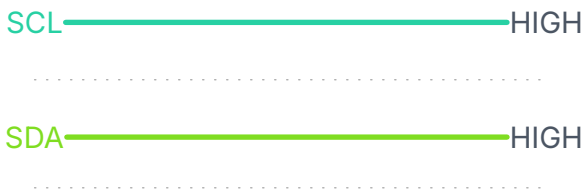


Figure 2: Idle bus

The resulting logic behavior can be viewed as:

Line state	Result
Nobody pulls LOW	HIGH
Someone pulls LOW	LOW

This means that a LOW state is “dominant” on the bus.

Bus Topology

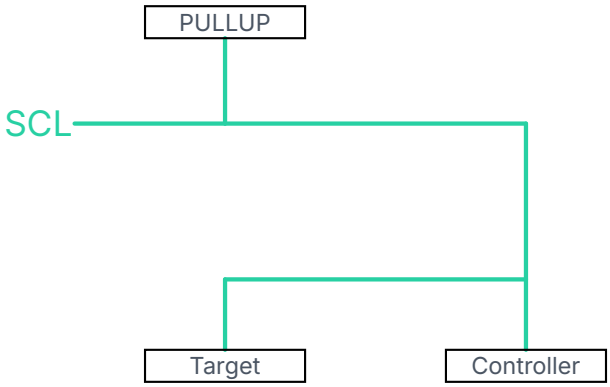


Figure 3: SCL wiring

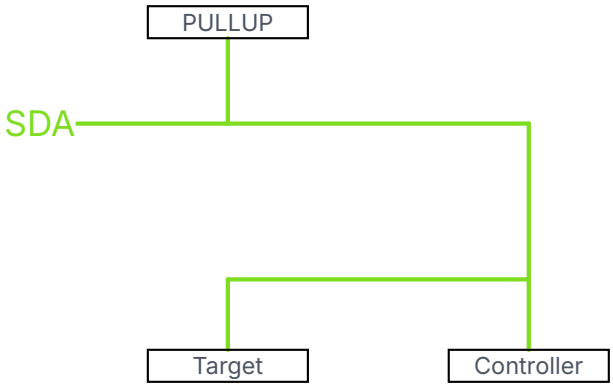


Figure 4: SDA wiring

All devices share the same SDA and SCL lines.

Data Transfer

Communication begins with a Start condition and ends with a Stop condition.
A Start condition occurs when SDA transitions from HIGH to LOW while SCL remains HIGH.



Figure 5: Start condition

A Stop condition occurs when SDA transitions from LOW to HIGH while SCL remains HIGH.



Figure 6: Stop condition

After the Start condition, the controller transmits a target address followed by data bytes. Data is transferred one bit at a time on SDA while SCL provides timing. SDA remains stable while SCL is HIGH and may only change while SCL is LOW.

Simplified example:

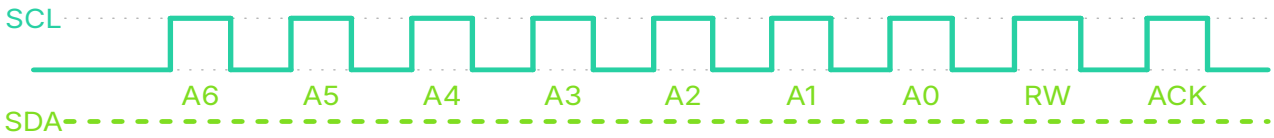


Figure 7: Address

Where:

A6..A0 represent the target address. R/W indicates a read or write operation. ACK is an acknowledgement bit generated by the receiving device.

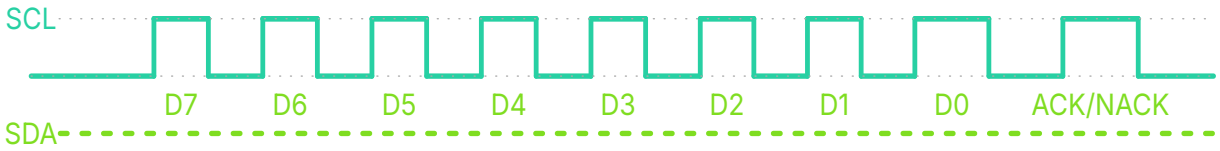


Figure 8: Data

D7..D0 represent the data sent over the bus. The *slave* device can respond with ACK/NACK to indicate their acknowledge status.

You can have multiple *master* on the same bus, but in reality it just means that at any point, someone takes the *master* role and all other *master*s becomes *slave*s. Since the communication is bidirectional on the same wire, only one person can talk at the same time

Advantages

This protocol only needs two-wire to talk to at most 112 devices. You can wire all of these devices in parallel without any additional wires.

Drawbacks

Every device needs its own address, and you can't have duplicated addresses otherwise things might behave strangely

SPI

SPI (Serial Peripheral Interface) is a simple, high-speed communication protocol commonly used to connect microcontrollers to peripheral devices such as sensors, displays, memory chips, and ADCs.

SPI uses a ***master-slave* architecture**. The *master* device controls communication by generating a clock signal and selecting which *slave* device should communicate.

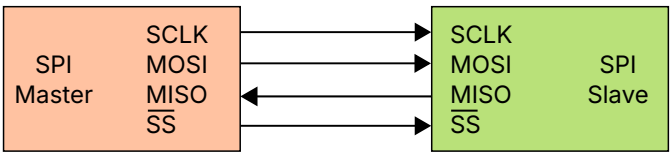


Figure 9: Standard wiring

SPI Signals

A standard SPI bus consists of four signals:

Signal	Name	Direction	Purpose
SCLK	Serial Clock	Output	Synchronizes data transfers
MOSI	Master Out, Slave In	Output	Data sent from *master* to *slave*
MISO	Master In, Slave Out	Input	Data sent from *slave* to *master*
CS (SS)	Chip Select (Slave Select)	Output	Enables a specific *slave*

The clock signal determines when data is sampled and shifted between devices. MOSI and MISO allow simultaneous transmission in both directions.

Full-Duplex Communication

Unlike protocols such as I²C, SPI is inherently **full-duplex**. Every clock pulse shifts one bit from the *master* to the *slave* and one bit from the *slave* to the *master* at the same time.

This means transmitting and receiving occur simultaneously, even if one side is only sending dummy data.

Data Transfer

Communication begins when the *master* asserts a *slave*'s Chip Select line. Once selected, the *master* generates clock pulses and both devices shift data through their internal shift registers.

After the desired number of bits have been transferred, the *master* deasserts Chip Select, ending the transaction.

SPI typically transfers data in 8-bit words, but many devices support other word lengths.

Clock Polarity and Phase

SPI devices must agree on how data is synchronized with the clock signal. Two settings define this behavior:

- **CPOL (Clock Polarity)** determines the idle state of the clock.
- **CPHA (Clock Phase)** determines which clock edge is used for sampling data.

Together these settings create four SPI modes:

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Master and *slave* must use the same mode for communication to work correctly.

Multiple Slaves

A single *master* can communicate with multiple *slave* devices.

The most common arrangement shares the clock and data lines between all *slave*s while giving each *slave* its own Chip Select signal. The *master* activates only one Chip Select at a time, ensuring that only the intended device responds.

Figure 10: Multi Drop Wiring

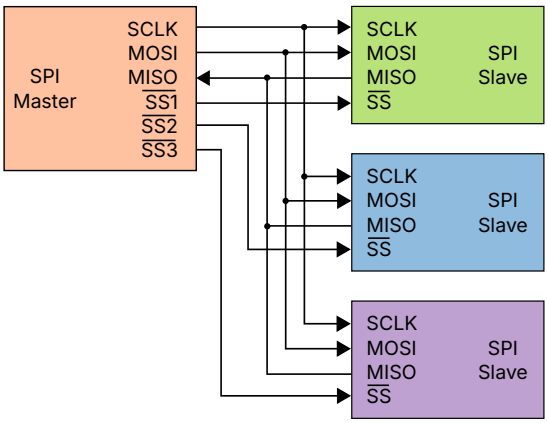
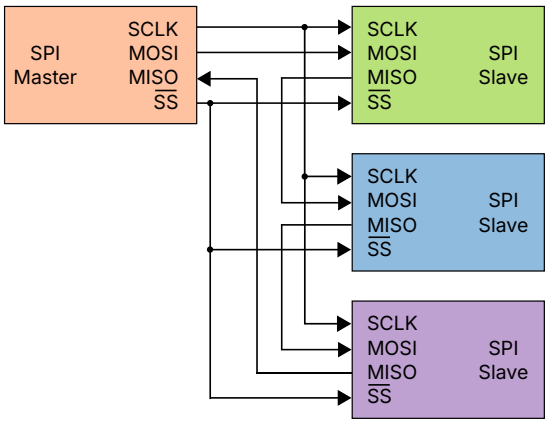


Figure 11: Daisy Chained Wiring



In the multidrop bus configuration, each *slave* has its own SS, and the *master* selects only one at a time. MISO, SCLK, and MOSI are each shared by all devices. This is the way SPI is normally used.

SPI can also be used in a daisy chain configuration, where the first *slave*'s output is connected to the second *slave*'s input, and so on with subsequent *slave*s, until the final *slave*, whose output is connected back to the *master*'s input. This configuration only requires a single SS line from the *master*, rather than a separate SS line for each *slave*.

Advantages

- Simple hardware implementation
- High communication speeds
- Full-duplex operation
- Low protocol overhead
- Widely supported by microcontrollers and peripherals

Limitations

- Requires more wires than I²C
- No built-in device addressing
- Typically supports only short-distance communication
- Each additional *slave* usually requires another Chip Select line

USB

Overview

The system communicates with a host computer using the Universal Serial Bus (USB) protocol. USB provides a standardized method for device discovery, configuration, and data exchange over a single cable.

We will use the **Human Interface Device (HID)** device class. HID is a widely supported USB class intended for user input devices such as keyboards, mice, game controllers, and similar peripherals. Because HID support is built into common operating systems, no custom driver is typically required.

The device presents itself to the host as one or more HID interfaces, allowing keyboard and mouse reports to be exchanged using the standard USB HID protocol.

Physical Layer

USB uses four conductors:

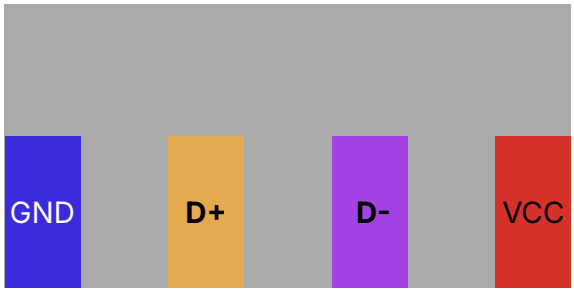


Figure 12: USB A male connector pinout

The **D+** and **D-** signals form a **differential pair**. Instead of measuring each wire independently, the receiver observes the voltage difference between the two wires. This improves noise immunity and allows reliable communication in electrically noisy environments.

The bus can be in several logical states:

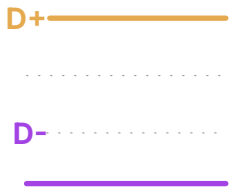


Figure 13: J State (Idle)

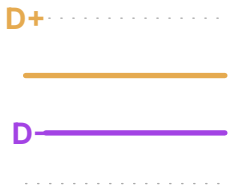


Figure 14: K State (Idle)

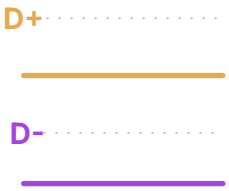


Figure 15: SE0 (Single-Ended Zero)

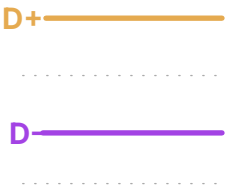


Figure 16: SE1 (Single-Ended One) - Illegal State

These states are used by the protocol to represent data, synchronization patterns, reset conditions, and end-of-packet markers. The exact voltage thresholds are defined by the USB specification and are handled by the USB transceiver hardware.

USB Device Enumeration

When the device is connected, the host performs a process known as **enumeration**. During enumeration the host:

1. Detects that a device is attached.
2. Resets the USB connection.
3. Requests identification information from the device.
4. Reads the device descriptors.
5. Assigns a unique USB address.
6. Configures the device for normal operation.

The descriptors provide information such as:

Vendor and product identifiers Device class information **HID report descriptions** Available interfaces and endpoints

Once enumeration is complete, the device is available for normal use.

USB HID Protocol

The HID class defines a standardized method for exchanging input and output reports.

A **report** is a structured packet of data whose format is described by a HID Report Descriptor. The descriptor tells the host how to interpret the report contents.

For a keyboard, a report commonly contains information such as:

Pressed key codes Modifier keys (Shift, Ctrl, Alt, etc.)

For a mouse, a report commonly contains:

Relative X movement Relative Y movement **Button states** Scroll wheel movement

Table 7: Example keyboard data

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Byte 0	RSuper	RAlt	RShift	RCtrl	LSuper	LAlt	LShift	LCtrl
Byte 1	*Reserved*							
Byte 2	*Keypress #0*							
Byte 3	*Keypress #1*							
Byte 4	*Keypress #2*							
Byte 5	*Keypress #3*							
Byte 6	*Keypress #4*							
Byte 7	*Keypress #5*							

Table 8: Example mouse data

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Byte 0	0	0	0	0	0	Middle	Right	Left
Byte 1	*ΔX Movement (8-bit signed integer)*							
Byte 2	*ΔY Movement (8-bit signed integer)*							

The actual report structure is defined by the HID Report Descriptor implemented by the device firmware.

Data Transfers

USB communication occurs through **endpoints**. An endpoint represents a logical communication channel between the host and the device.

HID devices typically use:

Control transfers for configuration and descriptor exchange. Interrupt transfers for periodic input reports.

Despite the name, interrupt transfers do not generate CPU interrupts on the USB bus itself. Instead, the host periodically polls the device to determine whether new input data is available.

The communication model is host-controlled:
The device only responds to host messages. It can't talk unless it was talked to

Keyboard Operation

When a key is pressed or released, the firmware updates the current keyboard state and generates a HID keyboard report.

The operating system interprets the report and converts the key code into the appropriate character or action according to the active keyboard layout.

Mouse Operation

Mouse movement and button events are reported using HID mouse reports.

Movement is generally represented as a change relative to the previous position rather than an absolute screen coordinate.

The host operating system accumulates these relative movements and updates the pointer position accordingly.

Final Words

Thanks for reading, we hope that you'll love our product when it is finished

Erratum

Splonkpad

For the splonkpad, we weren't able to implement a mouse substitute due to the fact that we didn't buy the vendor calibration tool and tried to calibrate by hand. The IC still works, but with mangled data.

With the 3d printed part, we can still show that the finger tracking isn't completly borked, as the data is printed to uart and there is a small python program that visualize it.

The finger tracking still only works on the Up/Down axis (correctly at least)